

Mandatory and Optional Deliverables

- Github Repo & Source Code
- Interactive Dashboard
- Cloud Deployment
- Assumptions & Decisions Log
- Summary: Completed Work
- Summary: Incomplete Work
- AI Usage Disclosure

Github Repo & Source Code - https://github.com/adith23/airbnb_market_intelligence

Interactive Dashboard: https://airbnb-dashboard-prod-605975162766.us-central1.run.app/Market_Overview

Cloud Deployment: https://airbnb-dashboard-prod-605975162766.us-central1.run.app/Market_Overview

1. Assumptions and Decisions Log

1.1 . Assumptions Log

- **Occupancy Inference** A calendar day marked as not available (indicated by f in the raw calendar data) is assumed to represent a paid booking. This assumption has a high impact because it can overestimate host occupancy and revenue, as hosts often manually block out calendar dates for personal travel, property cleaning, or scheduled maintenance. The mitigation is to treat all calculated occupancy and revenue figures as theoretical maximums and upper-bound estimates.
- **Annual Revenue Estimation** Estimated annual revenue is computed using the formula: Occupancy Rate multiplied by the Average Nightly Price multiplied by 365 days. The impact of this assumption is high because it relies directly on the inferred occupancy rate, making it an upper-bound yield estimate rather than a reflection of actual bank transactions.
- **Review Proxy for Booking Volume** The metric reviews per month is used as a proxy for booking frequency, assuming that approximately 50 percent of guests leave a review (modeled as reviews per month multiplied by 2). This assumption has a medium impact since it underrepresents absolute booking volumes by omitting guests who do not leave reviews, and it is therefore used exclusively for relative velocity comparisons between listings.

- **Amsterdam Pricing Fallback** The raw calendar CSV file for Amsterdam contains 100 percent null values in the daily price column. To prevent empty inputs in downstream pricing algorithms, the pipeline inherits the base nightly price from the listings detailed file. The impact of this is medium because it misses seasonal and weekend price fluctuations in Amsterdam, resulting in a flatter price trend.
- **Currency and Price Normalization** Nightly price strings contain local currency symbols (USD for NYC, Euros for Amsterdam and Barcelona) and formatting characters. The pipeline strips these markers and converts all values to a decimal float. Price metrics represent the nominal value in local currency at the time of scraping, and exchange rates are applied point-in-time for cross-city comparisons. The impact is low.

1.2 Technical Decisions Log

- **Data Processing Engine: Polars** We selected Polars instead of Pandas or PySpark for ETL processing. Polars uses vectorized, multi-threaded execution written in Rust, which performs columnar transformations significantly faster than Pandas. Its lazy evaluation allows processing of New York City's 12.7 million calendar rows on standard developer RAM without memory blowouts.
- **Analytical Database: DuckDB** We chose DuckDB over SQLite or PostgreSQL as our analytical database. DuckDB is an embedded, serverless database that runs in-process inside the Python runtime. Its columnar OLAP storage is optimized for large aggregations and can query Parquet files directly, whereas SQLite is OLTP-focused and PostgreSQL requires database server deployments.
- **Storage Format: Parquet** We selected Parquet as the intermediate storage format rather than CSV or JSON. Parquet is a column-oriented binary format that compresses files by up to 80 percent using dictionary encoding. It preserves explicit schemas and data types between pipeline runs and allows column pruning to speed up subsequent loads.
- **Predictive ML Model: XGBoost** We chose XGBoost over Ridge Regression or Random Forest. XGBoost is a best-in-class gradient-boosting algorithm for structured tabular data. It handles missing values natively, resists multicollinearity, and models non-linear spatial relationships, though it requires hyperparameter tuning to avoid overfitting.

- **Infrastructure: Docker and GCP Cloud Run** We selected containerized deployment using Docker and GCP Cloud Run instead of virtual machine deployments or AWS ECS. Standardized container builds ensure 100 percent replication of code across developer machines and cloud instances. Serverless scaling deploys the dashboard and pipeline runs on-demand, minimizing active resource costs.
- **Architecture Model: Medallion Star Schema** We chose a progressive Medallion architecture (staging data from Bronze to Silver to Gold and finally loading it into a DuckDB Star Schema) instead of flat wide tables. This structures raw tables into optimized, read-only OLAP entities with fact tables and dimension tables (dim_city, dim_host, dim_property, dim_neighbourhood, dim_date, and dim_reviewer) linked via explicit primary and foreign keys.

2. Summary of Completed Work

2.1 Core Objectives Achieved:

- **Robust Data Infrastructure (Data Engineering):** I designed and implemented an automated, configuration-driven ETL pipeline in Python and Polars that manages ingestion, schema profiling, and data quality validation against declarative rules. I standardized raw data into compressed staging Parquet files and constructed a high-performance local analytical database using a dimensional star schema in DuckDB.
- **Advanced Analytics and Storytelling (Descriptive and Spatial-Temporal):** I conducted rigorous exploratory data analysis (EDA) across three markets (New York City, Barcelona, and Amsterdam), generating summary statistics, geographic pricing gradients, and seasonal occupancy patterns. I built an interactive multi-page Streamlit business intelligence dashboard containing seven dedicated analytical modules, integrating spatial Folium/Kepler.gl maps and a conversational AI database assistant.
- **Predictive Analytics and Responsible AI (Data Science and ML):** I trained, optimized, and compared multiple regression model families (Ridge, Random Forest, LightGBM, and XGBoost) to predict nightly listing prices. The champion XGBoost Regressor achieved a Mean Absolute Percentage Error (MAPE) of 23.90% and an R-squared of 0.7114. I conducted SHAP-based feature driver interpretation, cross-city transfer matrix model generalizability checks, and demographic/geographic bias audits.

2.2 Key Features Implemented:

- **Automated Ingestion, Profiling, and Quality Reporting:** Pipelines for download (downloader.py), schema/outlier profiling (profiler.py), and validation (validator.py) with configurable constraints. Ingestion computes MD5 checksums of raw files to enable hash-based incremental load skips, logs schema drift history, and tracks automated data lineage.

- **Cleaning, Standardization, and Enrichment:** Implemented robust cleaning rules (`cleaner.py`) for local currency stripping, ISO 8601 date parsing, and property type consolidation. Custom feature engineering (`enricher.py`) computes derived metrics at listing and daily granularity, including inferred occupancy rate (from calendar availability status), estimated annual revenue, host tenure, and price per bedroom.
- **Relational Star Schema Database:** Constructed fact and dimension tables (`modeler.py`) in a local `airbnb.duckdb` file: dimensions include `dim_city`, `dim_host`, `dim_property`, `dim_neighbourhood`, `dim_date`, and `dim_reviewer`, with facts mapping listing snapshots, daily calendar schedules, and reviews.
- **Conversational AI SQL Assistant:** Developed a secure natural language interface within the Streamlit dashboard using Gemini 2.5 Flash. Built guardrails including a regex-based keyword blacklist to enforce read-only commands and automatic row-limit injection (`LIMIT 50`) to protect application memory from large database queries.
- **Statistical Inference Engine:** A reusable utility module (`stats_utils.py`) that performs D'Agostino-Pearson normality checks, non-parametric Mann-Whitney U testing (testing Entire Home vs. Private Room pricing and Superhost ratings), Kruskal-Wallis H tests for neighborhood pricing variance, and Wilcoxon signed-rank tests for weekday vs. weekend rate differences. Runs OLS regression on log-transformed prices with VIF collinearity filtering (beds dropped due to high VIF).

2.3 Quality Standards and Best Practices:

- **Comprehensive Unit Test Suite:** Written using `pytest` to validate statistical calculations, data loaders, and pipeline components (passing with 100% success rate).
- **Environment Replication and Containerization:** Configured `Dockerfile` and `docker-compose.yml` to ensure consistent local and cloud runtime execution.

3. Summary of Incomplete Work

3.1 Deferred Features:

- **dbt and Great Expectations Integration:** Reason for incomplete status: The assessment suggested using dbt or Great Expectations for transformation and data quality. This was deferred to avoid adding the compilation overhead of dbt and the heavy, complex dependency structure of Great Expectations. Prioritization Rationale: Prioritized writing a lightweight, self-contained custom validation engine in Polars and Python (`validator.py`) to keep the repository portable and ensure rapid local execution with zero external database compilation dependencies.

- Real-time Stream Processing Simulation: Reason for incomplete status: The core datasets provided by Inside Airbnb are monthly static snapshots, meaning real-time stream processing does not align with the characteristics of the source data. Prioritization Rationale: Prioritized batch pipeline optimization and Streamlit dashboard modules, deferring streaming simulations as a secondary value-add.
- Listing Cover Image Aesthetic Grading (CNN): Reason for incomplete status: Training a deep learning CNN model to grade the design quality of property photos would require downloading and processing gigabytes of image binaries. Prioritization Rationale: Prioritized tabular pricing model optimization (which explains over 71% of pricing variance) and statistical hypothesis testing due to the limited time and local computational resources.
- Review NLP Sentiment Topic Modeling (BERTopic/LDA): Reason for incomplete status: Running advanced sentiment and topic extraction models on reviews was deferred due to time limits. Prioritization Rationale: Prioritized developing the conversational Text-to-SQL AI database assistant to demonstrate high-value interactive database access for business analysts, deferring review topic modeling to a future phase.

3.2 Known Issues and Tech Debt:

- DuckDB Concurrent Write Locking: DuckDB is an in-process database and locks database writes during pipeline execution. This is currently managed by enforcing read-only connections in the Streamlit application during active ETL pipeline runs.
- Amsterdam Calendar Pricing Fallback: Amsterdam's daily calendar pricing relies on a static fallback to the listing's base price because the raw calendar feed contained 100% null values. This misses seasonal fluctuations in Amsterdam.
- AST SQL Parser for Guardrails: The conversational agent's security validation relies on a regex blacklist. This should be refactored into a formal AST SQL parser to prevent sophisticated SQL injection obfuscation.

4. AI Usage Disclosure

In accordance with the Expernetic AI Tools Usage Policy (Section 10.1 of the assessment guidelines), I am providing this complete disclosure of the artificial intelligence tools and assistants I integrated into my development workflow during this assignment.

4.1 AI Assistance Log

To maintain full transparency, this log documents the specific tools I used, how they assisted each phase, the key prompts I designed, and how I verified and modified the generated outputs.

- **AI Tools Used:** I utilized Cursor as my primary code editor, with Antigravity as the agentic coding assistant. I used Claude Opus 4.6 (Thinking Mode) for high-level statistical planning and architecture design, and Gemini 3.1 Pro (High) and Codex 5.5 (High) for code generation, debugging, and testing.
- **AI-Assisted Sections:** Every phase of this project - including the data engineering pipeline, statistical analysis, model training, and Streamlit dashboard - was AI-assisted. I used the AI to break down the requirements of each section, draft step-by-step implementation plans, write the initial boilerplate code, and review the resulting logic against the project's constraints.
- **Key Prompts Used:** (1) System Architecture: "Draft a comprehensive, modular architecture and data flow plan for this section, ensuring alignment with the project's data engineering constraints and model validation standards." (2) Code Generation: "Implement the following function adhering to production-grade coding standards, ensuring clear separation of concerns, exception handling, and strict typing." (3) Pipeline Integration: "Evaluate the architectural trade-offs between integrating the Exploratory Data Analysis (EDA) scripts directly into the main data pipeline versus isolating them in modular, local Jupyter Notebooks." (4) Dashboard Design: "Based on the target audience of product managers and revenue strategists, identify the key analytical insights, pricing metrics, and host segmentation views that should be prioritized for a Streamlit business intelligence dashboard." (5) Pipeline Debugging: "Diagnose potential bottlenecks or lock issues in a DuckDB pipeline that hangs during the compilation and loading of Parquet tables in run-pipeline-all after reverting modifications in the enrichment and modeling layers."
- **Output Validation:** I validated all AI-generated outputs by executing code locally in isolated environments, running unit tests (pytest), and performing manual checks on the data warehouse outputs. For the Text-to-SQL agent, I manually verified that the generated DuckDB SQL matched the intended star schema and did not trigger security exceptions.
- **Modifications Made:** The AI-generated ingestion code originally attempted to load both listings.csv.gz and the expanded listings.csv into memory simultaneously. This caused severe RAM spike issues and hung my development laptop during run-pipeline-all --skip-download. I modified the pipeline to load only listings.csv.gz since it already contains the complete set of required attributes. I also added robust database locks to the DuckDB connection manager to resolve transaction conflicts.
- **Critical Assessment:** I rejected the AI's initial recommendation to train a single global price prediction model. After reviewing the cross-city transfer matrix logs, which showed severe MAE inflation when transferring models across cities, I decided to deploy city-specific XGBoost models instead. I also rejected the AI's suggestion to use standard parametric Welch's t-tests for pricing analysis due to severe normality violations, implementing non-parametric Mann-Whitney U tests instead.

4.2 Workflow and Verification Rationale

- I did not treat AI as a replacement for engineering judgment. Every system design, statistical test, and database query was planned and reviewed step-by-step before any code was written. To ensure the reliability of the system, I validated the outputs by writing custom unit tests (such as `test_stats_utils.py`) to verify the accuracy of the scipy statistical calculations. I also inspected the metadata and data lineage tables in DuckDB to confirm that all column mappings and data enrichments matched my design.